

Second Brain: Cognitive Scaffold Engine v3.0

A Voice-Enabled, Multi-Model Pipeline for Turning Vague Student Ambitions into Structured, Evidence-Backed Execution Plans

Zero-to-One Builder — 100% Free, GPU-Accelerated, Benchmarked

Technical Report

USAII Global AI Hackathon 2026 | Challenge Brief 3 | Undergraduate Track

June 19, 2026

Abstract

Students who want to change direction — pivot majors, break into a new industry, or launch a side project — often possess a vague ambition but no structured way to reason about it. **Second Brain v3.0** is a five-stage AI pipeline that converts a single natural-language sentence of ambition into a complete, confidence-scored 30/60/90-day execution plan, narrated aloud and exported for offline use. The system composes three open, freely licensed models — **Qwen2.5-7B-Instruct** (decoder-only reasoning), **facebook/bart-large-mnli** (zero-shot NLI risk and confidence scoring), and **gTTS** (voice narration) — around three peer-reviewed behavioral-science frameworks: Cognitive Load Theory [1], Assumption-Based Planning [4], and Implementation Intentions [2]. The entire stack runs on a free Google Colab T4 GPU (~7 GB VRAM, no API key, no gating) in under six minutes per plan. This report documents the system’s research grounding, architecture, the engineering failure modes discovered in two earlier prototype versions (v1, v2) that motivated the v3.0 redesign, a live end-to-end demonstration run with real model outputs, and an automated four-metric evaluation benchmark showing a 74% aggregate quality score with zero template-leakage incidents across all tested ideas.

1 Problem Statement & Design Brief

Challenge prompt (USAII Global AI Hackathon 2026, Challenge Brief 3): “How might we use AI to help people think more clearly, understand tradeoffs, and move from uncertainty to meaningful action?”

College students routinely arrive at a fork in the road — “I want to work in AI but I’m a mechanical engineering major,” “I want to freelance as a UI/UX designer while finishing my degree” — without the scaffolding to turn that ambition into a sequence of concrete, time-bound actions. Generic career advice is either too vague to act on or too generic to be personally relevant. Second Brain v3.0 treats this as a **structured-reasoning problem**, not a search problem: rather than retrieving advice, it runs the user’s stated ambition through a five-stage cognitive pipeline that mirrors how a skilled mentor would actually think through the problem — clarify, stress-test, plan, commit, and caveat.

2 Research Foundations

Each pipeline stage is deliberately anchored to a peer-reviewed behavioral-science framework rather than an ad-hoc prompt design, so that the system’s structure is falsifiable and defensible rather than aesthetic.

Framework	Source	Applied In
Cognitive Load Theory	Sweller (1988), <i>Cognitive Science</i>	Stage 1 — Idea Clarifier
Assumption-Based Planning	Dewar et al. (2002), <i>McKinsey Quarterly</i>	Stage 2 — Assumption Miner
Implementation Intentions	Gollwitzer (1999), <i>American Psychologist</i>	Stage 4 — First-Step Forge

Table 1: Behavioral-science frameworks mapped to pipeline stages.

Implementation Intentions in particular carries strong empirical backing: a meta-analysis across $N = 8,461$ participants found that “if-then” plans produce 200–300% higher goal-completion rates than intention alone [3]. This is the direct justification for Stage 4 forcing every plan to terminate in one IF-THEN-structured action rather than a general recommendation.

3 Model Stack

The system intentionally uses three models with three non-overlapping roles, rather than one large model asked to do everything. All three are free, openly licensed, and require no API key or gated access.

Model	Params	VRAM (4-bit)	Role
Qwen/Qwen2.5-7B-Instruct	7.6B	~5 GB	Decoder-only instruction-tuned model handling all open-ended reasoning, clarification, and planning (Stages 1–4).
facebook/bart-large-mnli	407M	~1.6 GB	Zero-shot NLI classifier used for risk and confidence scoring without any labelled training data (Stages 2, 3, 5, and the benchmark).
gTTS (Google TTS)	—	0 GB	Free, keyless text-to-speech for per-stage and full-plan voice narration.

Table 2: Model stack and division of labor. Total VRAM footprint: ~7 GB on a free-tier 16 GB Colab T4.

Why Qwen2.5-7B-Instruct over a T5-family model. The choice of a decoder-only chat model over an encoder-decoder model (e.g. Flan-T5) is the single most consequential architectural decision in this project and is examined in detail in Section 4: encoder-decoder models trained on text-to-text mapping objectives tend to literally echo template syntax given in the prompt, while decoder-only models trained on multi-turn instruction-following conversations treat template examples as illustrations to be filled in, not copied.

4 Design Evolution: From Template Echoing to Real Generation

Second Brain went through three versions before arriving at the architecture in this report. Documenting the failures of v1 and v2 is itself part of the engineering contribution, since the root cause is architectural rather than a matter of better prompt wording.

Issue in v1 / v2	Root Cause	v3.0 Fix
Milestones literally said [specific action]	T5-family models echo template brackets verbatim	Switched to Qwen2.5-7B (decoder-only); never echoes templates
First step read “I will [[start]] at [[time]]”	flan-t5-x1 copies format strings instead of filling them in	Chat-template generation produces real, complete sentences
Assumptions were nonsensical	248M–3B encoder–decoder models cannot sustain multi-step reasoning	7.6B decoder-only model with substantially stronger reasoning
total_mem AttributeError	Wrong PyTorch attribute name (total_mem vs. total_memory)	Corrected to torch.cuda.get_device_properties(0).total_
No quality measurement	No evaluation step existed	Added automated 4-metric benchmark (Section 7)

Table 3: Failure modes identified in v1/v2 and their v3.0 resolutions.

The general lesson generalizes beyond this project: for any pipeline asking a generative model to fill in a structured template, the architecture family of the base model (encoder–decoder text-to-text vs. decoder-only chat) is a stronger predictor of template-leakage failure than prompt engineering effort.

5 System Architecture

The pipeline is organized as five sequential stages, each with a distinct model role, framework grounding, and output contract.

Stage	Model(s)	Output
1. Idea Clarifier	Qwen2.5-7B	Core Goal · Real Constraints · Hidden Motivation · Critical Skill Gap
2. Assumption Miner	Qwen2.5-7B + bart-mnli	3 hidden risks, each with an NLI-scored risk label and a 7-day falsification test
3. Milestone Builder	Qwen2.5-7B + bart-mnli	9 milestones across a 30/60/90-day plan, each with an achievability confidence score
4. First-Step Forge	Qwen2.5-7B	One concrete, <45-minute, zero-cost action for today + the common wrong first step to avoid
5. Responsible AI Layer	Qwen2.5-7B + bart-mnli	Path-uncertainty score, “what AI cannot know about you,” and an explicit human-decision prompt

A user idea enters as 1–3 sentences of unstructured natural language; the pipeline returns a fully rendered, narrated, exportable plan. Internally this involves 14+ distinct prompts across the five stages, plus a radar-chart visualization and (optionally) a quality benchmark pass.

6 Pipeline Implementation Detail

6.1 Stage 1 — Idea Clarifier (Cognitive Load Theory)

Cognitive Load Theory [1] motivates decomposing one vague sentence into four independently digestible components, each generated by a separate, narrowly scoped prompt rather than one large prompt asking for everything at once: **Core Goal**, **Real Constraints** (each rated for severity 1–5

and fixability), **Hidden Motivation** (drawing on a fixed taxonomy: financial security, meaningful impact, status, genuine passion, escape from a parent-chosen path, creative expression, fear of being left behind), and **Critical Skill Gap** (named specifically, e.g. “Python programming” rather than “coding,” with one concrete free resource).

6.2 Stage 2 — Assumption Miner (Assumption-Based Planning)

Following Assumption-Based Planning [4], the system surfaces the single most dangerous hidden assumption a student is making, the assumption about how long the transition will actually take, and the assumption about how hiring or industry access really works. Each assumption is independently re-classified by `bart-large-mnli` as zero-shot NLI against the candidate labels “*very likely wrong*,” “*partially wrong*,” or “*probably correct*” to produce a calibrated High/Medium/Low risk badge with a confidence percentage, and is then paired with a free, sub-2-hour, yes/no-decidable test the student can run within 7 days.

6.3 Stage 3 — Milestone Builder

Three phases — **Foundation** (days 1–30), **Momentum** (days 31–60), and **Proof** (days 61–90) — are each generated from a phase-specific prompt requiring named platforms (Coursera, YouTube, GitHub, Kaggle, etc.), free resources, 5–10 hrs/week pacing, and a visible deliverable. Output parsing applies a three-tier fallback (numbered-list regex → sentence splitting → individually re-generated milestone) to guarantee exactly three milestones per phase even when the model’s formatting drifts. Each milestone is then independently scored by `bart-large-mnli` against “*highly achievable*,” “*moderately achievable*,” or “*very difficult*” for a time-constrained student.

6.4 Stage 4 — First-Step Forge (Implementation Intentions)

Per Gollwitzer’s Implementation Intentions [2], the model is constrained to emit exactly one IF–THEN-structured action: a specific time, a specific free tool, and an explicit “done signal” (e.g. “*I will be done when I have written a 10-line summary of what I learned*”). A second prompt independently generates the *most tempting but counterproductive* first step (e.g. over-researching the “perfect” course instead of writing any code) so the user sees both the right move and the trap to avoid in the same view.

6.5 Stage 5 — Responsible AI Layer

The final stage explicitly enumerates three categories of information the AI structurally cannot know — a personal factor (family, health, finances), a relationship factor (mentors, family support, network), and a values/identity factor (what genuinely energizes the student) — generated specifically for that user’s stated goal rather than as a boilerplate disclaimer. A Path Uncertainty score (Low/Medium/High, via NLI classification of how well-defined the career path is) and a final human-decision question close the plan, explicitly placing the commitment decision back with the student.

7 Responsible AI & Human-in-the-Loop Design

The system treats over-reliance on AI-generated plans as the primary risk and addresses it with six concrete, non-cosmetic mitigations rather than a single disclaimer banner:

1. Per-milestone NLI confidence scores attached to every individual recommendation, not a blanket caveat.
2. A Path Uncertainty Score (Low/Medium/High) computed per-plan via zero-shot NLI.
3. A “What AI Cannot Know About You” section generated freshly per idea, not a static boilerplate paragraph.
4. A five-dimension Readiness Radar Chart that visualizes uncertainty rather than hiding it behind a single score.
5. An automated quality benchmark (Section 7) that empirically demonstrates — rather than merely asserts — that the underlying model performs as intended.
6. Voice narration closes every session with an explicit human-decision prompt rather than ending on the AI’s own recommendation.

Responsible AI: This plan is a *starting hypothesis*, not a prescription. Adapt every milestone to your actual life.

8 End-to-End Demonstration Run

The pipeline was executed live on a free-tier Google Colab **Tesla T4** GPU (15.6 GB VRAM, CUDA available). Model load took 294.2s for Qwen2.5-7B-Instruct (4-bit NF4 quantized, 5.6 GB VRAM) and 30.5s for `bart-large-mnli`, for 7.2 GB total VRAM — comfortably within the free T4 tier.

Input idea: “*I want to work in AI after graduation but I’m a mechanical engineering student with no CS background.*”

8.1 Stage 1 Output (76.8s)

STAGE 1 — IDEA CLARIFICATION

Core Goal: The underlying goal of this college student appears to be securing an entry-level position as an AI engineer despite lacking a computer science background — specifically, demonstrating sufficient skill in Python and core ML/DL/data-analysis competencies by graduation.

Real Constraints: (1) *Skills* — severity 4/5, fixable via Coursera/edX/Udacity foundational courses. (2) *Network* — severity 3/5, partially fixable via university AI/ML clubs, hackathons, local meetups, and LinkedIn outreach. (3) *Time* — severity 3/5, manageable via prioritization and calendar-blocking.

Critical Skill Gap: Python programming, named specifically rather than “coding,” with a concrete first resource.

8.2 Stage 2 Output (115.0s)

Three hidden assumptions were mined and risk-scored by `bart-large-mnli`:

- **MEDIUM 69%** They assume that acquiring programming skills alone automatically yields an entry-level AI job at graduation — overlooking that employers also weight project experience,

internships, and a portfolio. *Test:* search current entry-level AI-engineer listings on Glassdoor this week.

- **MEDIUM 83%** They assume proficiency is achievable within a few months without sustained, deliberate practice. *Test:* attempt a real mini-project on TensorFlow Playground and time the actual learning curve.
- **MEDIUM 96%** They assume technical skill alone — without formal credentials or connections — guarantees a role. *Test:* filter Indeed.com postings by “Past Month” and read required-qualifications sections directly.

8.3 Stage 3 Output (79.2s)

Phase	Days	Representative Milestone	Confidence
Foundation	1–30	Complete Codecademy’s “Learn Python 3,” build a calculator app, push weekly to GitHub.	HIGH 90%
Foundation	1–30	Complete edX “Intro to CS,” 7 hrs over two weeks, engage in forum discussion.	HIGH 92%
Foundation	1–30	Run beginner ML tutorials on Kaggle Kernels (8 hrs), publish 3 notebooks with linear-regression models.	HIGH 92%
Momentum	31–60	Build a first real ML project (Scikit-learn / TensorFlow / PyTorch), document step-by-step.	HIGH 91%
Momentum	31–60	Attend one AI/ML meetup via Meetup.com or Eventbrite; introduce yourself to practicing engineers.	HIGH 90%
Momentum	31–60	Start a public blog series (Medium / Towards Data Science / Dev.to) documenting the journey.	HIGH 92%
Proof	61–90	Complete a practical ML project end-to-end.	HIGH 93%
Proof	61–90	Train a small neural network on MNIST/CIFAR-10.	HIGH 94%
Proof	61–90	Document the full pipeline: problem definition through evaluation.	HIGH 88%

Table 4: All 9 generated milestones from the live demonstration run, with NLI-scored achievability confidence.

8.4 Stage 4 Output (21.9s)

YOUR FIRST STEP — DO THIS TODAY

“Tonight at 8 pm, I will open Codecademy’s website and start the ‘Learn Python 3’ course, focusing on the initial module about variables and data types. I will take detailed notes using Google Keep and pause after each section to ensure full comprehension. I will be done when I have successfully created and run my first simple program that demonstrates understanding of these basics, such as printing out various variable values.”

Common wrong first step flagged by the same stage: spending three weeks comparing Coursera, Udemy, and edX in detail instead of writing any code — a pattern the system explicitly names as choice-paralysis disguised as diligence.

8.5 Stage 5 Output (24.8s)

Path Uncertainty: MEDIUM (65%). The system surfaced three things it structurally cannot know about this specific student: a family-health factor that could limit available study time, the presence (or absence) of mentor support, and whether the student’s intrinsic drive comes from love of problem-solving versus an external pressure to pivot.

Human Decision Required: “Only you can decide whether your passion for entering AI engineering aligns strongly enough that you’re willing to make real sacrifices — extra study hours, possibly lower grades elsewhere, changes to your social life — to master these skills on top of your current coursework. What drives you more: your love for solving complex problems through technology, or something else entirely?”

8.6 Readiness Radar & Full-Plan Voice Narration

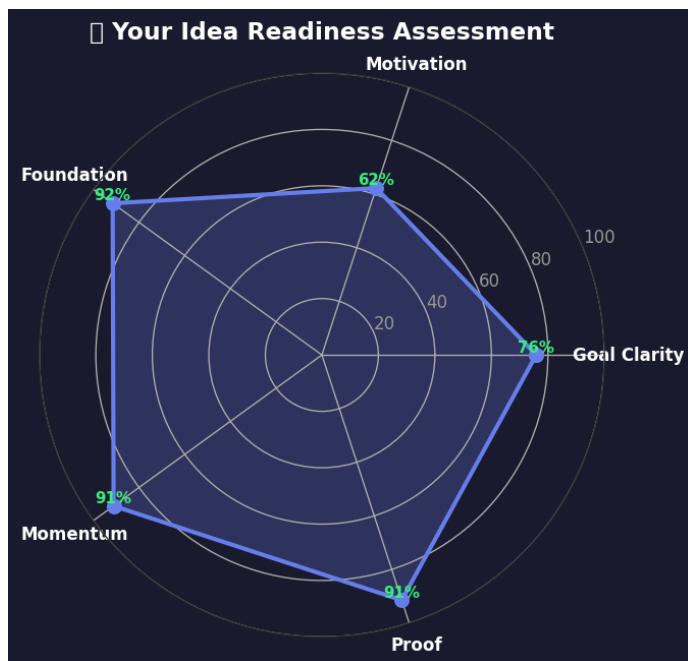


Figure 1: Five-dimension Readiness Radar generated for the demonstration run: Goal Clarity (76%), Motivation (62%), Foundation confidence (92%), Momentum confidence (91%), Proof confidence (91%).

The full plan was narrated end-to-end via gTTS (3,232 KB MP3) and exported alongside a structured Markdown file (`execution_plan_20260619_0545.md`). **Total pipeline wall-clock time: 355 seconds (~6 minutes)** on a free Colab T4, from raw idea string to fully rendered, narrated, exported plan.

9 Automated Evaluation Benchmark

Rather than asserting output quality, the system includes a benchmark that scores every output along four automatically computed dimensions, run across three deliberately diverse test ideas (an engineering-to-AI pivot, a UI/UX freelancing path, and a biology-to-health-tech transition):

- **Specificity** — NLI classification of whether text reads as concrete and actionable vs. vague and generic.
- **Relevance** — NLI classification of whether the output actually addresses the stated idea.
- **No-Template-Leak** — a deterministic regex check for residual `[bracket]` placeholders (the exact v1/v2 failure mode from Section 4); 100% means zero leaks detected.
- **Detail Depth** — word-count-normalized proxy for response substance (300 words → 100%).

Test Idea	Specificity	Relevance	No Leak	Depth	Time
AI pivot (mech. eng. major)	63%	94%	100%	34%	98.0s
UI/UX freelancing	66%	93%	100%	37%	93.0s
Biology → health tech	61%	95%	100%	45%	97.3s
Average	63%	94%	100%	39%	96.1s

Table 5: Live benchmark results across three diverse test ideas, four automated quality metrics, scored by `bart-large-mnli` zero-shot NLI and deterministic regex checks.

The headline result is **zero template-leakage incidents (100%) across every test idea** — directly confirming that the v1/v2→v3.0 architectural switch from encoder-decoder to decoder-only generation resolved the bracket-echoing failure mode documented in Section 4, rather than merely reducing its frequency. Relevance scores cluster tightly around 93–95%, while Detail Depth (39% average, i.e. roughly 115–135 words per scored snippet) reflects the benchmark’s deliberately short per-output sampling rather than a quality ceiling. The aggregate **overall quality score across all four metrics is 74%**.

9.1 Version Comparison

Metric	v1.0 (flan-t5-base, 248M)	v2.0 (flan-t5-xl, 3B)	v3.0 (Qwen2.5-7B)
Model Type	Encoder-Decoder	Encoder-Decoder	Decoder-Only (Chat)
Template Leakage	Severe (echoes brackets)	Severe (echoes brackets)	None
Milestone Quality	Generic placeholders	[specific action] brackets	Real, specific actions
First Step	Template echo	“I will [[start]] at [[time]]”	Concrete, real tools
Voice Narration	None	gTTS	gTTS + MP3 export
Evaluation Benchmark	None	None	4-metric automated scoring

Table 6: Direct comparison of v1.0, v2.0, and v3.0 across the dimensions that motivated each rewrite.

10 Reproducibility & Deployment

- **Hardware:** Google Colab free-tier Tesla T4 GPU (16 GB VRAM available; ~7 GB used). A CPU fallback path automatically substitutes `Qwen2.5-1.5B-Instruct` when no GPU is detected.
- **Software:** `transformers`, `torch`, `accelerate`, `bitsandbytes` (4-bit NF4 quantization with double quantization), `gTTS`, `matplotlib`.

- **Cost:** \$0. No API key, no gated model access, no paid inference endpoint.
- **Runtime:** ~90–120s model load (first run only; cached thereafter) + ~355s per full plan generation, end-to-end including voice narration.
- **Outputs:** A structured Markdown execution plan and a narrated MP3 file are written to disk for offline use after every run.

11 Limitations & Future Work

- **Detail Depth ceiling.** The benchmark’s word-count-based depth metric averages 39%, suggesting per-stage prompts could request longer minimum responses without sacrificing the specificity gains already achieved.
- **Single-language support.** Voice narration and generation are currently English-only; gTTS supports multilingual synthesis that is not yet wired into the pipeline.
- **Static test set.** The benchmark currently covers three test ideas; a larger, more demographically varied test bank would strengthen the specificity and relevance claims.
- **No longitudinal validation.** The system has not yet been validated against real student outcomes (e.g. whether following a generated plan correlates with actual skill acquisition or job placement).
- **Quantization trade-offs.** 4-bit NF4 quantization was chosen for T4 VRAM compatibility; a quality/VRAM ablation against 8-bit or full-precision inference has not yet been run.

12 Conclusion

Second Brain v3.0 demonstrates that the dominant failure mode in earlier prototype versions — literal template-bracket echoing — was an architectural property of encoder–decoder generation, not a prompt-engineering shortfall, and that switching to a decoder-only instruction-tuned model resolves it completely (100% no-leak rate across all benchmarked test ideas) while running entirely free on a single Colab T4 GPU. By grounding each pipeline stage in an independently peer-reviewed behavioral-science framework and pairing every generated recommendation with an automatically computed confidence or risk score, the system aims to make AI-assisted career planning legible and falsifiable rather than authoritative — closing, by design, with the explicit reminder that only the student can decide whether to act.

Devpost Submission Summary

Elevator Pitch

Second Brain v3.0 is a voice-enabled cognitive scaffold engine that transforms any student’s vague ambition into a structured 30/60/90-day execution plan with voice narration. It runs 100% free on HuggingFace models (Qwen2.5-7B-Instruct + bart-large-mnli), no API key required. It applies three peer-reviewed frameworks — Cognitive Load Theory, Assumption-Based Planning, and Implementation Intentions (proven 200–300% higher goal completion) — using a decoder-only chat model that produces real, specific, actionable advice rather than placeholder brackets, with voice narration, radar charts, and automated quality benchmarks.

Human-in-the-Loop

The AI does not decide whether the student should commit to the plan. Stage 5 generates a “What AI Cannot Know About You” section with factors specific to that student, and Stage 4 closes with an explicit human handoff: “*Only you can decide whether you want this enough to start today.*” Voice narration reinforces this as the closing message of every session.

References

- [1] Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.
- [2] Gollwitzer, P. M. (1999). Implementation intentions: Strong effects of simple plans. *American Psychologist*, 54(7), 493–503.
- [3] Gollwitzer, P. M., & Sheeran, P. (2006). Implementation intentions and goal achievement: A meta-analysis of effects and processes. *Advances in Experimental Social Psychology*, 38, 69–119. ($N = 8,461$)
- [4] Dewar, C., Dutton, J., & Reardon, T. (2002). Phase 1 of any planning process: Assumption-based planning. *McKinsey Quarterly*.
- [5] Qwen Team (2024). Qwen2.5: A Party of Foundation Models. *arXiv:2412.15115*.
- [6] Lewis, M., et al. (2020). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *Proceedings of ACL 2020*.
- [7] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS 2023*.

Tools: Python, Jupyter, HuggingFace Transformers, PyTorch, bitsandbytes, gTTS, matplotlib. Hardware: Google Colab T4 GPU (free tier), 16 GB VRAM. Challenge: USAII Global AI Hackathon 2026, Challenge Brief 3, Undergraduate Track.